Project Workflow – A Comprehensive Measure of Well-Being

The **A Comprehensive Measure of Well-Being** project is developed using a structured machine learning workflow consisting of multiple epics. Each epic represents a specific phase of the project, from environment setup and data preprocessing to model training, evaluation, and deployment. This systematic approach ensures the project is organized, scalable, and easy to maintain while enabling accurate prediction of the **Human Development Index (HDI)** using socio-economic indicators.

Epic 1: Environment Setup and Package Installation

Story 1: Install Required Software and Libraries

Install all the required Python packages, machine learning libraries, and Flask dependencies needed for data analysis, visualization, model development, and deployment. These include **NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, Flask, and Pickle**.

Story 2: Create the Project Folder Structure

Create and organize the project directory with dedicated folders for the dataset, training notebook, Flask application, templates, static resources, documentation, and output files. A well-structured folder hierarchy simplifies project maintenance and avoids path-related issues.

Epic 2: Importing Required Libraries

Story 1: Import Python Libraries

Import all necessary libraries such as **NumPy**, **Pandas**, **Matplotlib**, **Seaborn**, **Scikit-learn**, **Pickle**, and **Flask**. These libraries provide functionality for data manipulation, visualization, machine learning model development, model serialization, and web application deployment.

Epic 3: Dataset Download and Understanding

Story 1: Download and Load the Dataset

Download the **Human Development Index (HDI)** dataset from Kaggle and load it into the Python environment using the Pandas library.

Story 2: Explore the Dataset

Analyze the dataset by examining its dimensions, column names, data types, summary statistics, and target variable. Verify the dataset structure to understand the available socio-economic indicators used for predicting HDI.

Story 3: Perform Data Visualization

Generate visualizations such as strip plots, scatter plots, histograms, and a correlation heatmap to study the relationships between features including **Life Expectancy**, **Mean Years of Schooling**, **Expected Years of Schooling**, **Gross National Income (GNI) Per Capita**, and the **HDI Score**. These visualizations help identify trends and influential predictors.

Epic 4: Data Preprocessing and Feature Preparation

Story 1: Select Independent and Dependent Variables

Select the most relevant independent variables:

- Life Expectancy
- Mean Years of Schooling
- Expected Years of Schooling
- Gross National Income (GNI) Per Capita

Select **HDI Score** as the dependent (target) variable.

Story 2: Handle Missing Values

Check the dataset for missing or null values and replace missing numerical values using **mean imputation**. Verify that the dataset is complete before training the model.

Story 3: Encode Categorical Variables

Apply **Label Encoding** only if categorical columns such as **Country** are included in model training. If only numerical features are used, this step confirms that no encoding is required.

Story 4: Prepare the Dataset

Create the final cleaned dataset by selecting the required features, handling missing values, and ensuring all input variables are in a suitable numerical format for machine learning.

Epic 5: Dividing the Dataset into Train and Test Data

Story 1: Split the Dataset

Split the preprocessed dataset into **training** and **testing** datasets using Scikit-learn's `train_test_split()` function. The training set is used to build the Linear Regression model, while the testing set is reserved for evaluating the model's performance on unseen data.

Epic 6: Building and Evaluating the Linear Regression Model

Story 1: Train the Model

Train a **Linear Regression** model using the prepared training dataset. The model learns the relationship between the selected socio-economic indicators and the Human Development Index (HDI).

Story 2: Generate Predictions

Use the trained model to predict HDI values for the testing dataset and compare the predicted values with the actual HDI scores.

Story 3: Evaluate Model Performance

Evaluate the model using regression performance metrics such as:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score (Coefficient of Determination)

Visualize the comparison between actual and predicted HDI values to assess prediction accuracy.

Epic 7: Saving the Trained Model

Story 1: Serialize the Model

Save the trained Linear Regression model as a **Pickle** (`.pkl`) file. Serialization allows the model to be reused without retraining.

Story 2: Store the Model for Deployment

Store the generated **HDI.pkl** file in the Flask application directory so that it can be loaded efficiently during prediction through the web application.

Epic 8: Building the Flask Web Application

Story 1: Develop the Flask Backend

Develop the Flask application to:

- Accept user inputs for socio-economic indicators.
- Load the trained Linear Regression model.
- Process the inputs.
- Generate and display the predicted Human Development Index (HDI).

Story 2: Create the User Interface

Design an intuitive web interface using **HTML**, **CSS**, and optionally **JavaScript**. Integrate the templates with Flask to provide an interactive prediction form and display the predicted HDI score.

Story 3: Test and Deploy the Application

Run the Flask application locally, verify prediction accuracy, test different input combinations, and ensure that all application components function correctly. Confirm that the application provides reliable HDI predictions and a smooth user experience.
